

Lab 2 - Introduction to FHIR Resources

- Due Monday by 8am
- Points 8
- Submitting an external tool
- Available Jan 12 at 8am - Feb 9 at 8am

Lab 2 - Introduction to FHIR Resources

Warning: Many students find this to be the most difficult lab. Please be mindful of this when planning your schedule.

Overview

This lab will utilize Gradescope for grading. To get started, read the assignment instructions below and then follow the link on the left hand menu to Gradescope when you are ready to submit your code. Please read the entire assignment description and the steps carefully before getting started.

This lab runs through six (6) exercises involving FHIR Resource structure and general concepts as well as working with Bundles, with a focus on **exploration** of FHIR and HAPI FHIR. The lab consists of two Java class files, ResourceHandler and BundleHandler which will be what you submit via Gradescope. (You will have unlimited submissions. However, you should **not** attempt to use these submissions to troubleshoot your code, instead you should consider implementing tests in the local framework provided.)

- **Part I:** Consists of the first four exercises (1-4) and will be based around the FHIR Patient Resource, Observation Resource, and the US Core Patient Profile. The methods you will implement range from simple to moderately complex, with a focus on understanding data structures using the HAPI FHIR library. These exercises will be implemented in the ResourceHandler Class.
- **Part II:** Consists of two exercises (5-6), which will be focused on Bundles, including navigation and understanding meta information as well as how to work with resources contained within the bundles. These exercises will be implemented in the BundleHandler Class.

For all parts of this lab you should only be using the HAPI FHIR classes (models and client) with basic Java classes such as Lists/Arrays/Sets as needed. **You should not need to use anything complex. Additional libraries will not be available on Gradescope.**

Learning Objectives

- Understand the structure of FHIR Resources, including how to handle list/array elements and FHIR data types.
- Understand how to manipulate resources to perform simple tasks.
- Understand how paged bundle navigation is accomplished.

Tools Used

- Gradescope
- Any IDE of your choice. (IntelliJ is recommended for Java.)
- Java 1.8
 - Note: Other Java versions should be fine for your local starter code if you do not use more recent features including lambdas, but the Java autograder is locked to 1.8 and we cannot modify it at this time.

Resources

- Lab Starter Code: <https://github.gatech.edu/cs6440-students/Lab-2-Introduction-to-FHIR-Resources>  (<https://github.gatech.edu/cs6440-students/Lab-2-Introduction-to-FHIR-Resources>)
- [HAPI FHIR Primer \(Also found in course files under content\)](https://gatech.instructure.com/courses/499698/files/70288307?verifier=S1fMsLD7ElgjIVsBOWjQ0L1cnSRjDB7sO1GQeGMG&wrap=1)
(<https://gatech.instructure.com/courses/499698/files/70288307?verifier=S1fMsLD7ElgjIVsBOWjQ0L1cnSRjDB7sO1GQeGMG&wrap=1>) 
(https://gatech.instructure.com/courses/499698/files/70288307/download?verifier=S1fMsLD7ElgjIVsBOWjQ0L1cnSRjDB7sO1GQeGMG&download_frd=1) - Details HAPI FHIR Java implementation and how to use it, based on this lab. (Note: The primer includes a reference to the parser being not allowed. This does **not** apply to testing locally, though you should not use it in any of your submitted code--there is no need to as all resource will be passed to your methods.)
- [IntelliJ IDE](https://www.jetbrains.com/idea/download/)  (<https://www.jetbrains.com/idea/download/>)
 - You may qualify for a free version of the Enterprise Edition as a GT Student. Make sure to use your GaTech email address.
- [FHIR Specification](https://hl7.org/fhir/R4/index.html)  (<https://hl7.org/fhir/R4/index.html>) ([Patient Resource](https://hl7.org/fhir/R4/patient.html)  (<https://hl7.org/fhir/R4/patient.html>) | [Observation Resource](https://hl7.org/fhir/R4/observation.html)  (<https://hl7.org/fhir/R4/observation.html>) | [US Core Patient Profile](http://hl7.org/fhir/us/core/structuredefinition-us-core-patient.html)  (<http://hl7.org/fhir/us/core/structuredefinition-us-core-patient.html>))
 - **Note:** Make sure you are on R4!

- **HAPI FHIR** [↗\(https://hapifhir.io/\)](https://hapifhir.io/) (**General Documentation** [↗\(https://hapifhir.io/hapi-fhir/docs/\)](https://hapifhir.io/hapi-fhir/docs/)) | **JAVA Docs** [↗\(https://hapifhir.io/hapi-fhir/docs/appendix/javadocs.html\)](https://hapifhir.io/hapi-fhir/docs/appendix/javadocs.html))
 - **Note:** For the purposes of this lab and the JAVA Docs, you will mostly be interested in the R4 Structures found at: <https://hapifhir.io/hapi-fhir/apidocs/hapi-fhir-structures-r4/> [↗\(https://hapifhir.io/hapi-fhir/apidocs/hapi-fhir-structures-r4/\)](https://hapifhir.io/hapi-fhir/apidocs/hapi-fhir-structures-r4/). For the classes based around standard resources, you will want to click on "org.hl7.fhir.r4.model" and then look under the Class Summary list for what you are after. This list includes far more than just the primary resources themselves, so you will likely want to do a search on the page for the specific resource you are looking for (e.g. "Patient").

Getting Started

1. **Watch** the FHIR related lectures for the week. **Lectures 8.1 and 8.2 are required for this lab.**
2. **Read** the HAPI FHIR Primer and any HAPI FHIR documentation pages noted in the primer, also found under the resources header below.
3. **Download** the lab starter code. While you are free to use an IDE of your choice, IntelliJ is strongly recommended and all instructions for setup will be provided for it. **You may find the lab starter code here:** <https://github.gatech.edu/cs6440-students/Lab-2-Introduction-to-FHIR-Resources> [↗\(https://github.gatech.edu/cs6440-students/Lab-2-Introduction-to-FHIR-Resources\)](https://github.gatech.edu/cs6440-students/Lab-2-Introduction-to-FHIR-Resources). You may either download the repo as a zip and extract it locally or clone it from command line using: "**git clone https://github.gatech.edu/cs6440-students/Lab-2-Introduction-to-FHIR-Resources**". For more information on what is included in the starter code and how to use it, please see the Starter Code section under resources.
4. **Read** through the Exercise write-ups found below thoroughly. They provide not only general instructions but important pointers.
5. **Explore** the FHIR Specification site and the HAPI FHIR library. Keep in mind the exercises provided only give you partial information by design, you are intended to locate what you need in the specification and HAPI FHIR documentation.
6. **Complete** the ResourceHandler exercises 1-4, using the provided lab starter code. All of your code should be in the ResourceHandler.java file.
7. **Complete** the BundleHandler exercises 5-6, using the provided lab starter code. All of your code should be in the BundleHandler.java file.
8. **Submit** your ResourceHandler.java and BundleHandler.java files to Gradescope. **You may submit to Gradescope as many times as required.** For more information on submitting your assignment to Gradescope, please see the section at the bottom of this page.

WARNING: Gradescope is not intended for troubleshooting your code. There is no provided output that will indicate the exact nature of your error in most cases. The only output you will be given is a comparison of expected values (typically strings) when the

assert fails. This is only for ensuring you do not get hung up on minor issues like case sensitivity or typos. Output regarding the *structure* of the Resources is omitted, as understanding the structure of the resources by using the documentation is the primary purpose of the lab.

Exercises

Part I - Exercises 1-4

Exercise 1 - Get Resource ID

The goal of this exercise is to create a method that returns the ID of a resource as a String, without any additional details such as the base URL or resource type.

EXAMPLE: Given a full URL of a resource as "http://someserver.com/fhir/Patient/12345", you would only return 12345.

Exercise 2 - Set Observation Code

For exercise 2, you will create a convenience method that sets the code (referring to a key/attribute in the object, `observation.code`) element of an Observation resource. Your method is provided the Observation resource object to modify, and strings for the system, code (referring to the standardized medical code, e.g. LOINC), and display. You are then returning the Observation (as in the object) with the code element set. The purpose of this exercise is understanding the structure of Coding and CodeableConcept types in FHIR and how they are nested within a resource. (**NOTE:** The function in the starter code and on Gradescope is technically misnamed, you are *SETTING* not adding. You should **not** modify the starter code to reflect this naming inconsistency, simply write your Java code with this in mind.)

Exercise 3 - Update Patient Official Given Name

For this exercise, you will create a more practical method focused on updating a patient's official given name. This is a common case where someone has their legal name changed and it must be updated in a system. (Though real world implementation may vary around maintaining records of past names, for our purposes we will imagine a scenario in which only one official name is maintained.) For your method signature you will be provided a Patient resource and a String of the new name. You must navigate the resource and **replace** the existing given name(s) with the provided name. Please note that you are provided with a

single string and do not need to consider additional given names (for example, middle names). No other values in the resource should be modified except the official given name. If your approach replaces the entire name, it will fail Gradescope.

EXAMPLE: If your patient has a full name of "Bob Smith", and you are passed the replacement given name string "Robert", you must maintain the existing family name of "Smith". Your method should thus return a Patient object with a full name of "Robert Smith".

Exercise 4 - Create US Core Patient

This exercise is to create a Patient resource using the US Core Patient profile. Please be sure to review the lectures for information on profiles. To avoid redundant complexity, you will only be required to work with a handful of core pieces of demographic data present in the base Patient resource along with the US Core Ethnicity extension. As with profiles, please be sure to review the lecture material on extensions. Your method will be provided all of the required information and then should return a **newly created** Patient resource. In addition to the provided method arguments, you **must** review the FHIR specification for both the US Core Patient profile and US Core Ethnicity Extension in order to identify key fixed values you must set, such as the actual profile meta tag and the Ethnicity URLs (which includes both the top level extension URL in the form of a structure definition address, as well as additional nested extension URLs for each part of the extension which are simple descriptive strings).

Part II - Exercises 5-6

Exercise 5 - Bundle Navigation

This exercise involves handling navigation requests for a bundle. Your method will be provided two arguments, a Bundle of resources and a HAPI FHIR String constant indicating previous or next links. You should take these arguments and based on the link constant provided (which it is strongly recommended to use in your own code for comparison) attempt to return the URL for the requested page of the bundle. If a link is not present (such as for a request for the next page while on the final page of a bundle) then you will return a string representation of "-1". (Note: While it is not standard practice to return a string of "-1", it is used here as a simple approach for the purposes of this assignment while maintaining the String return type, without adding additional complexity.)

Implement your code inside the "navigateBundle()" method in the BundleHandler class between the comments noting the location to place student code. The starter code includes a line at the end that returns an empty string to ensure the project compiles properly prior to your implementation, you can disregard this.

EXAMPLE: Suppose the test class calls the method as "navigateBundle(bundle, Bundle.LINK_PREV)" where the bundle object provided is the first page of the full bundle. As there is no previous page to navigate to, the method would return the String "-1".

Presuming the bundle has multiple pages and the call to the method instead had Bundle.LINK_NEXT as the second argument, it would return a String containing the URL of the following page as provided in the bundle which would begin with something similar to "http://some-fhir-endpoint?_getpages=...".

Exercise 6 - Obtaining Resources from Bundles

For the final exercise you will demonstrate navigating the resources inside a Bundle in order to achieve a goal. Here you will look at the Patient resources within a provided **(complete, non-paged)** Bundle and return a Java ArrayList of the patients who are flagged as deceased. You will need to review the FHIR Specification for the Patient resource to get an idea of what to look for here.

Implement your code inside the "getListOfDeceasedPatients()" method in the BundleHandler class between the comments noting the location to place student code. Add your entries to the already instantiated list called "patientArrayList" which will be returned by the method. While it is unlikely to be an issue for the majority of possible implementations, you will want to ensure that your patients are listed in patientArrayList in the same order in which they appear in the bundle.

HINT: You do not need to be concerned with the time they were deceased, only whether or not they are. As "deceased" is a choice of data type element, this leaves you with 4 potential cases shown below.

Data Type	Value	Is Patient Deceased?
<i>Neither deceasedBoolean nor deceasedDateTime is present.</i>	<i>None</i>	No, though technically this means unknown the implication is the patient is not deceased.
deceasedDateTime	Any DateTime	Yes, the presence of any date implies deceased. (A date in the future would be a data entry error and not prophetic hopefully.)
deceasedBoolean	True	Yes, the patient is deceased but date/time is not known.

deceasedBoolean	False	No, the patient is explicitly marked as not deceased.
-----------------	-------	---

Starter Code

Repository Link: <https://github.gatech.edu/cs6440-students/Lab-2-Introduction-to-FHIR-Resources>  (<https://github.gatech.edu/cs6440-students/Lab-2-Introduction-to-FHIR-Resources>)

What is included in the Starter Code?

The starter code consists of a very basic Maven project intended for use with IntelliJ. It includes templates for the ResourceHandler and BundleHandler classes you will be implementing, and the framework to test them locally before submission to Gradescope. It also includes templates for unit tests for these two classes, as well as a sample implementation of FHIR Resource JSON parsing you can use for testing (do **not** parse anything in your actual methods). In addition to the classes and tests, it includes several resource JSON files which you can use to test against. These files can be found in the "/src/resources/" directory.

Is the Starter Code required?

While strictly speaking you are free to implement the code without an IDE if you want a challenge for some reason, you are still required to utilize at least the two classes provided in the starter code. Gradescope will require the package and method signatures to remain consistent to function.

Opening the Starter Code in IntelliJ

After downloading (and extracting) or cloning the repo, open the project's root directory in IntelliJ. This will be the one that includes the pom.xml file. IntelliJ **should** be smart enough to recognize this as an indication the project uses Maven, but sometimes this doesn't seem to be the case (if you aren't familiar with Maven, don't worry about it, it just helps manage dependencies). If you have any errors listed in the project, you will want to open the project view in IntelliJ (the branching directory structure usually in a left hand window pane) and right click on the pom.xml file and select "Add as Maven Project". This may additionally require right clicking on the pom.xml file again and selecting "Reload Project". This should resolve any missing imports. If you have any issues with the starter code, please post your questions on Ed.

WARNING: Gradescope is locked to Java Version 1.8 and cannot be updated at this time. Please use it accordingly. In IntelliJ, you can confirm this version under "Preferences -> Build, Execution, Deployment -> Compiler -> Java Compiler" and then look at the lab module's target bytecode version. While you can technically use a more recent version

and not worry about 1.8 locally, you need to make sure you are not using Java features added after it. This includes lambda expressions.

Using the Testing Framework

The testing framework provided in the starter code provides you the opportunity to test your code locally against a resource known to be valid. The tests demonstrate how to parse the provided JSON files into valid FHIR Resource objects. (This is done precisely the same way in the autograder to ensure consistency and avoid human error.) Using these resource objects, you can provide tests for your code. Precise tests as found on Gradescope will not be included. Please note that for the US Core Patient profile exercise, you will essentially be rebuilding the resource in its entirety. The test case comes with a pre-parsed Patient resource using the US Core profile you can use for exploration and writing a test case. It is recommended to see how you might extract the needed arguments for your method from the object and then use the parser to do a side by side comparison of the JSON representation of the original parsed resource with your returned, newly created resource.)

Additional Notes

Provided imports and Adding new Imports

Please be aware that you cannot add any additional imports not from HAPI FHIR or the basis Java libraries or you will cause Gradescope to fail. All imports required to complete the assignment are provided for you, and you should not need to in any case.

FHIR Context and the Client

While the concepts around setting a context and using the client are important and discussed in the material, you do not need to worry about those for this assignment. This lab is entirely self contained and does not actually connect to a remote FHIR server.

Testing Locally and Gradescope Output

It is strongly recommended that you build local tests in the provided framework in the starter code. Testing against Gradescope is not productive due to the time it takes to execute Java containers, and it will not give you useful feedback on the structure of your results for many exercises. The output is left on in order to help identify minor inconsistencies around **values**. In other words, if you place the data in the wrong place, it is just going to tell you that your string is empty or it found a null object. However, if you have the data in the correct place but have some minor typo, particularly around case sensitivity, the grader will show the mismatch. The hope here is to prevent

you from getting hung up on issues that are not related to the goal of the assignment, even if technically required.

Submission

For general instructions, refer to the Gradescope help center at

https://help.gradescope.com/article/ccbpppziu9-student-submit-work#submitting_code  (https://help.gradescope.com/article/ccbpppziu9-student-submit-work#submitting_code). To begin, click on Gradescope on the left hand menu in Canvas.

Following the general instructions in the link provided above, click on the assignment shown. For this assignment you will only have the option to upload your code to Gradescope, so you will need to follow the file selection instructions appropriately and disregard the general submission instructions for a GitHub or Bitbucket submission. Your submission dialog window should look like the following. (Note that empty methods are being submitted for this example, so your file size will differ. Keep in mind as well that your code does not have to be fully implemented to attempt a submission to Gradescope, each test is executed individually and does not care about completeness, allowing you to do partial tests if you would like.) **(NOTE: The screenshot below shows an incorrect filename, please disregard. Both files should be titled "__Handler".)**

Submit Programming Assignment

 Upload all files for your submission

SUBMISSION METHOD

 Upload

Add files via Drag & Drop or [Browse Files](#).

NAME	SIZE	PROGRESS	
BundleHandler.java	0.7 KB		
ResourceBuilder.java	1.4 KB		

SUBMITTING FOR

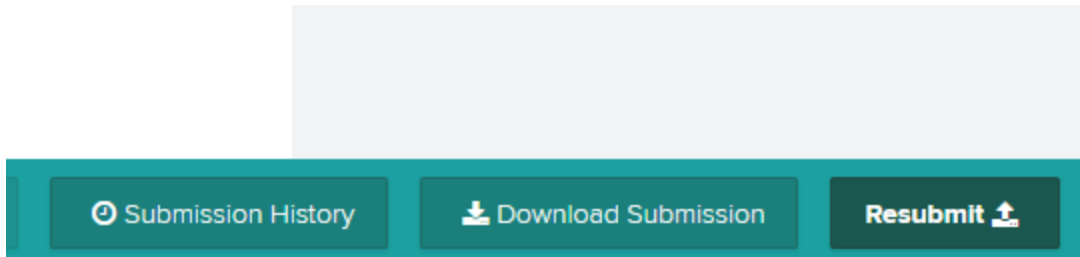
Elizabeth Margaret Shivers

Upload

Cancel

Once you make your submission, the grader will begin to run. Please give it a few minutes to complete, particularly in times of expected server stress such as coming up to the assignment due date. Your results will be displayed once complete. On the left hand side of the window it will show each test in detail (though passing solutions will not produce extraneous output) while the right hand side offers a summary view.

In order to resubmit your code you will find a button on the bottom right corner of the screen that can be easily overlooked. (Note: The student view may not look identical to what is provided here.)



WARNING: Gradescope allows you to select your best submission, but you must do so manually. Once you are satisfied with your submission, you must ensure that you have the proper submission "activated" in order for the grade to populate correctly to Canvas later. You can see the Submission History button in the previous screenshot. Provided below is an example of what this would look like.

Submission History

#	SUBMITTED AT (EST)	SUBMITTERS	SCORE	ACTIVE
5	Jan 05 at 9:01AM	EMS	0.0	Activate
4	Jan 04 at 4:50PM	EMS	5.0	✓
3	Jan 04 at 4:46PM	EMS	0.0	Activate
2	Jan 04 at 2:26PM	EMS	0.0	Activate
1	Jan 04 at 2:25PM	EMS	3.0	Activate

✕ Close

If you have any questions about Gradescope not covered here, please feel free to post them to the course forums.

There is nothing to submit in Canvas. You will be submitting all of your solutions via Gradescope for this lab.

Submit Programming Assignment

 Upload all files for your submission

Submission Method

 Upload

Drag & Drop

Any file(s) including .zip. Click to browse.